7. OpenClaw Gateway User Manual

The Gateway is the resident process of OpenClaw: it is responsible for maintaining the connections of the message channels, it carries the control and the event plane, and it acts as the unified entry point for the sessions, the routing and the channel state. Put simply: whether you can "connect to OpenClaw", whether the console can open and whether the messages can come in very often depends on whether the Gateway is running normally.

7. OpenClaw Gateway User Manual

1. When You Have to Care About the Gateway
2. The Fastest Way to Start It (local)
3. Which "External Interfaces" Does the Gateway Provide
4. The Hot Reload and the Changes That "Need a Restart"
5. The Authentication and the Remote Access (the easiest place to run into trouble)
    - 5.1 The Gateway Authentication (token/password)
    - 5.2 The Recommended Way of the Remote Access: tunnel first and then access
    - 5.3 The Console Does Not Open over Plain HTTP (device identity required)
6. The Multiple Instances and the "Rescue Robot" Mode
7. The Troubleshooting Order, It Can Locate Almost Everything
    - 7.1 Look at the Status First
    - 7.2 The Gateway Does Not Start: the Configuration Validation Fails
    - 7.3 The Common Symptom: the Service Is Installed but It Is Actually Not Running

1. When You Have to Care About the Gateway

In most cases you only have to let it run stably; you only have to check or learn the gateway in the scenarios below:

- The console often reports an error when you open it in the browser on the Jetson and you are about to switch to the TUI or to the remote access
- You changed the configuration but it "does not feel like it took effect" and you suspect that the hot reload/restart was not triggered
- You want to open the console to the LAN or to access it remotely through SSH/Tailscale
- You want a multi instance isolation (for example a "rescue robot" / redundancy)
- A port conflict, the process does not start, the service "looks like it is installed but it is not running"

2. The Fastest Way to Start It (local)

Run the following on the gateway host (for example the Jetson):

```
openclaw gateway
```

The common parameters:

```
# Print more complete debug/trace information to the current terminal (convenient for troubleshooting)
openclaw gateway --port 18789 --verbose

# When the port is occupied, try to terminate the listener that occupies the port and force the start
openclaw gateway --port 18789 --force
```

The port priority (from high to low): `--port` > `OPENCLAW_GATEWAY_PORT` > `gateway.port` > the default `18789`.

3. Which "External Interfaces" Does the Gateway Provide

The same port (18789 by default) provides the WebSocket control plane and the HTTP service (the control interface, the hooks, the A2UI and so on) at the same time; this is a "single port multiplexing".

The places you will commonly use:

- The local console: `http://127.0.0.1:18789/`
- The OpenAI Chat Completions compatible interface: `/v1/chat/completions`
- The OpenResponses interface: `/v1/responses`
- The tool invocation interface: `/tools/invoke`

In addition, the gateway starts the Canvas static file service by default (port 18793 by default), which serves the editable HTML/A2UI resources (served from `~/.openclaw/workspace/canvas` by default). To disable it, set `canvasHost.enabled=false` or `OPENCLAW_SKIP_CANVAS_HOST=1`.

4. The Hot Reload and the Changes That "Need a Restart"

The gateway watches `~/.openclaw/openclaw.json` (or the path given by `OPENCLAW_CONFIG_PATH`), and a configuration update is **usually applied automatically**. The default reload mode is `gateway.reload.mode="hybrid"`: the safe changes are applied hot and the critical changes trigger a restart.

You can configure the reload behaviour explicitly:

```
{
  "gateway": {
    "reload": { "mode": "hybrid", "debounceMs": 300 }
  }
}
```

Suggestions from experience:

- When you change the business configuration such as the model, the agent and the routing, you usually do not have to restart manually
- Changing `gateway.*` (the port, the binding, the authentication, the TLS, the HTTP and so on) is an infrastructure change and is more likely to trigger a restart

5. The Authentication and the Remote Access (the easiest place to run into trouble)

5.1 The Gateway Authentication (token/password)

The gateway has the authentication enabled by default: you can set `gateway.auth.token` (or `OPENCLAW_GATEWAY_TOKEN`) or `gateway.auth.password`. The client has to carry it in `connect.params.auth.token/password` when it connects.

If you use the wizard flow, a token is usually generated by default (even if it is only bound to the loopback).

5.2 The Recommended Way of the Remote Access: tunnel first and then access

The most recommended: Tailscale/VPN. Next: the SSH tunnel.

Example (map the remote 18789 to the local 18789):

```
ssh -N -L 18789:127.0.0.1:18789 user@host
```

Then you access it locally:

- Web: `http://127.0.0.1:18789/`
- WS: `ws://127.0.0.1:18789`

Note: even when you go through the tunnel, if the gateway has a token configured, the client still has to carry the token in order to connect.

5.3 The Console Does Not Open over Plain HTTP (device identity required)

If you open the console on the LAN with `http://<lan-ip>:18789/`, the browser may be in a non-secure context, so WebCrypto is restricted and the device identity cannot be generated, which results in `device identity required` / `connect failed`.[^gateway_troubleshooting]

The preferred repair routes:

- Open it locally: `http://127.0.0.1:18789/`
- Use Tailscale Serve to provide HTTPS in the remote scenario
- When you have to use HTTP, enable `gateway.controlUi.allowInsecureAuth: true` and use the gateway token (the token mode only, without the device identity/pairing)[^gateway_troubleshooting]

6. The Multiple Instances and the "Rescue Robot" Mode

> Only try this operation when you need it

Usually one gateway per host is enough; it is only recommended to run several gateway instances when you need redundancy or a strong isolation (for example a rescue robot).

The core principle of the multiple instances is "a full isolation + no conflict":

- A different `gateway.port`
- A different `OPENCLAW_CONFIG_PATH`
- A different `OPENCLAW_STATE_DIR`
- A different workspace (`agents.defaults.workspace`)

With the dev configuration file, you can quickly start a completely isolated development instance without affecting the main environment:

```
openclaw --dev setup
openclaw --dev gateway --allow-unconfigured
openclaw --dev status
openclaw --dev health
```

7. The Troubleshooting Order, It Can Locate Almost Everything

7.1 Look at the Status First

Run the following in order:[^gateway_troubleshooting]

```
openclaw status
openclaw status --all
openclaw status --deep
```

The commonly used additional commands:

```
openclaw gateway probe
openclaw channels status --probe
openclaw gateway status
openclaw logs --follow
openclaw doctor
```

7.2 The Gateway Does Not Start: the Configuration Validation Fails

The OpenClaw configuration is validated against a strict schema: an unknown key, a wrong type or an invalid value can all make the gateway refuse to start.

When the validation fails, usually only the diagnostic commands are available (for example `openclaw doctor / logs / health / status`). It is recommended to run the following directly:

```
openclaw doctor
```

7.3 The Common Symptom: the Service Is Installed but It Is Actually Not Running

If you install the gateway as a service with systemd/launchd/schtasks and so on, showing that it is "loaded/installed" does not mean that the process is running. Look at the following first: [^gateway_troubleshooting]

```
openclaw gateway status
openclaw logs --follow
```